STAR: A Decoupled Agent Framework for Industrial IEC 61131-3 Structured Text Synthesis

Benchmark Construction, Scan-Aligned Formal Verification, and Controlled CODESYS V3.5 Evaluation

Generative AI Course Final Project

14 November 2026

Abstract

Automatic synthesis of IEC 61131-3 Structured Text (ST) for industrial programmable logic controllers (PLCs) differs sharply from general-purpose software generation: candidate programs must conform to deterministic scan-cycle execution, retain internal edge and timer state across cycles, enforce same-scan safety preemption, and satisfy platform-specific CODESYS V3.5 compiler and static-analysis contracts. Moreover, code that compiles and passes finite test suites can still conceal single-scan mode-transition delays or priority regressions. This work presents an end-to-end engineering methodology and full 128-run empirical evaluation spanning three pillars: (1) a frozen 32-task benchmark (`normalized-20261114`, `dataset_sha256: d305f96c...`) combining 20 verbatim public tasks (`PUB001`–`PUB020`) and 12 AI-assisted, human-reviewed independent synthesis tasks (`SELF001`–`SELF012`), paired with 276 held-out fixed test cases (2,994 scan steps), 294 end-of-scan-aligned LTL specifications, and positive/negative dual-control `.smv` verification (32/32 reference models proven true; 32/32 mutant controls refuted); (2) **STAR**, a decoupled, leak-free agent harness combining a four-stage scan contract planner, relevance-ranked dialect cards, hash-guarded compiler repair, assertion-isolated in-loop symbolic verification (`Scheme A`, `ST->SMV` over `NuSMV` BDD and `nuXmv` SMT), and anti-lazy lexicographic ratchet simulation repair; and (3) a symmetric 2×2 benchmark comparing unmodified `Codex CLI v0.142.5` (**E1**, **E2**) against **STAR** (**E3**, **E4**) across `deepseek-v4.1-flash-1017` and `gpt-6.1-terra` under uniform maximum reasoning effort (`reasoning.effort = "high"`) and shared quotas. On `deepseek-v4.1-flash-1017`, **STAR** (**E3**) eliminates dialect collisions, elevating final compilation from **E1**'s 28/32 (87.5%) to **32/32 (100%)**, increasing evaluated fixed-case passes from 196/208 (94.2%) to **226/238 (95.0%, 81.9% of all 276 planned cases)** and task-level passes from 23/32 to **27/32**, while compressing cumulative context input nearly 11-fold (4.74M down to 433k tokens). On `gpt-6.1-terra`, unmodified `Codex CLI` (**E2**) achieves 29/32 (90.6%) compilation, the highest evaluated case pass rate (**220/222 = 99.1%**), 27/32 task pass, and 18 Stage 3 formal `PASS` verdicts with compact outputs (72.8k tokens, ¥5.21), whereas **STAR** (**E4**) matches **18** formal passes and 98.2% (217/221) evaluated cases using fewer inputs (366k tokens) but higher structured output cost (¥7.38) alongside three pre-patch compiler-lock timeouts. Finally, symbolic checking uncovers single-scan transition and exclusion defects in both **E3** (`PUB011`, passing 8/8 fixed tests) and **E1** (`PUB020`, passing 11/11 fixed tests), empirically validating dual runtime and formal verification.

1 Introduction and Problem Statement

1.1 Industrial Control Background and Core Challenges

Programmable logic controllers (PLCs) execute sequential interlocks, mode arbitration, timing supervision, and continuous regulation across manufacturing lines using IEC 61131-3 languages, with Structured Text (ST) serving as the primary high-level textual notation. Unlike stateless function evaluation in general software engineering, controller correctness depends on cyclic scan-cycle semantics and strict safety guarantees:

1. **Single-Scan Sampling and Persistent State**: Each `FUNCTION_BLOCK` invocation samples external inputs once per cycle, reconciles them against retained pre-scan internal state (edge memories, latches, timers), and updates outputs deterministically. Built-in stateful blocks such as `TON`, `TOF`, `TP`, `R_TRIG`, and `F_TRIG` must execute unconditionally on the top-level scan path rather than inside conditional branches where skipped calls freeze internal elapsed timers or edge histories.
2. **Same-Scan Safety Preemption and Dialect Contracts**: Emergency stops and protection interlocks must override motor or actuator commands within the exact scan cycle they assert. In addition, industrial deployment requires satisfying target compiler rules and static-analysis checks (`CODESYS V3.5 SP22 Patch 4`), including caller-owned `VAR_IN_OUT` composite storage, `ARRAY[*]` variable-length bounds, 0-based `STRING` element indexing, nanosecond `LTIME` literals, and `VARIANT` reflection.
3. **Insufficiency of Compilation and Finite Testing**: Large language models frequently emit syntactically valid ST code that compiles cleanly and passes representative test cases, yet fails on unobserved edge combinations—such as asserting a manual start signal on the exact rising scan where operating mode switches from manual to automatic.

1.2 Research Objectives and Scope Alignment

Table 1 maps the core engineering objectives and deliverables of this project to their implemented artifacts and current evaluation status.

Table 1: Alignment of core project objectives, artifacts, and verification status as of 14 November 2026.

Project Dimension	Core Technical Requirement	Implemented Artifacts & Current Status
Benchmark & LTL Specifications	Curate official + independent tasks (≥ 10), preserve provenance, verify scan semantics	**Completed**: 32 tasks (20 verbatim public + 12 AI-generated, human-reviewed self-built); 276 cases, 2,994 scans, 294 LTLs; 32/32 reference and 32/32 mutant SMVs verified.
STAR vs. Baseline Frameworks	Compare self-designed harness against established tool-using baseline across ≥ 2 models	**Completed**: Decoupled **STAR** harness vs. unmodified `Codex CLI v0.142.5` evaluated on `deepseek-v4.1-flash-1017` and `gpt-6.1-terra` at max reasoning.
Simulation & Formal Tooling	Integrate real PLC compilation/simulation and symbolic model checking	**Completed**: Real CODESYS SP22 compilation + 1 ms cyclic simulation, 5-stage `ST->SMV` compiler (`NuSMV` BDD / `nuXmv` SMT), `ScanContract`, and WebUI.
Controlled Empirical Evaluation	Execute 2×2 crossed matrix (128 runs), stratified analysis, deep-dive case studies	**Completed (128/128 Runs)**: All four groups (**E1**–**E4**, 32 tasks each) archived under unified `batch.py` adjudication with cross-condition quantitative and case analysis.
Cross-Platform Boundary	Optional extension toward Siemens TIA Portal or secondary targets	**Scoped Boundary**: Deep validation focused on `CODESYS V3.5 SP22 Patch 4`, with decoupled backend interfaces reserved for future targets.

2 Execution Environment, Toolchain, and Controlled Setup

2.1 Software Toolchain and Runtime Isolation

All compilation and functional test runs execute against a real Windows installation of `CODESYS Development System 3.5.22.40` (`CODESYS V3.5 SP22 Patch 4`) driven via its embedded IronPython 2.7 Scripting Engine, with `CODESYS Static Analysis Light` active. Candidate function blocks are built

inside copies of a hash-frozen template project (`codesys/templates/benchmark.project`, device target `Control Win V3 x64`, `template_sha256: 70d40be7b4b9bdf4a22619a7ba47156422cf6beaec692480f0e5ee2acbda8909`). Runtime functional evaluation drives the compiled `FB_Task` under a `1ms` cyclic `ScriptTaskObject` running a generated `PLC_PRG` harness over the local simulation target; no external physical PLC hardware is connected.

Formal verification is dispatched dynamically across two symbolic model checkers: **NuSMV 2.7.1** (BDD engine) for pure Boolean and bounded integer controllers, and **nuXmv** (with the `MathSAT` SMT backend, default BMC depth 25) for designs containing IEEE-754 floating-point ports (`REAL`/`LREAL`) or unbounded arithmetic. Software orchestration and offline regression suites run on Python 3.12+ inside a standard environment, while real CODESYS compilation and gateway-connected simulation run inside normal un-sandboxed Windows processes because restricted sandboxes block APInstaller event-log writes and gateway communication.

2.2 Experimental Matrix and Reasoning Configuration

Following `configs/fullflow_matrix.json`, evaluation uses a crossed 2×2 design across two frameworks—the self-developed **STAR** harness (`self_harness`) and unmodified official **Codex CLI v0.142.5**—and two foundation models accessible through the project relay (`deepseek-v4.1-flash-1017` and `gpt-6.1-terra`), totaling 128 planned task runs across 32 benchmark tasks (Table 2). Crucially, **all four conditions uniformly enforce maximum reasoning strength** (`reasoning.effort = "high"` in every API payload), isolating architectural and model differences without varying inference effort. Pre-configured ablation modes (`no_planner`, `no_dialect`, `no_selftest`) are additionally preserved in `configs/experiments.json` for follow-up attribution studies.

Table 2: Controlled 2×2 benchmark matrix spanning agent frameworks and foundation models.

ID	Agent Framework	Model Route	Reasoning Effort & Runtime Transport
E1	**Codex CLI (v0.142.5)**	`deepseek-v4.1-flash-1017`	Max (`reasoning.effort = "high"`), local `CodexTransport` HTTP relay + MCP server
E2	**Codex CLI (v0.142.5)**	`gpt-6.1-terra`	Max (`reasoning.effort = "high"`), local `CodexTransport` HTTP relay + MCP server
E3	**STAR (`self_harness`)**	`deepseek-v4.1-flash-1017`	Max (`reasoning.effort = "high"`), native multi-stage `ResponsesClient`
E4	**STAR (`self_harness`)**	`gpt-6.1-terra`	Max (`reasoning.effort = "high"`), native multi-stage `ResponsesClient`

2.3 Shared Budgets, Quotas, and Engineering Guardrails

Rather than treating operational limits as optimal universal constants, runtime quotas act as symmetric guardrails for fairness and cost containment across agents. As logged in each 14 November 2026 run's `generation/runtime_config.json`:

- **Symmetric Tool and Call Quotas**: Each task allows at most `max_repairs = 2` compile-repair iterations (initial draft plus up to 2 repairs, i.e., ≤ 3 builds), `max_verifications = 3` formal checks (capped at `60s` each), `max_simulations = 3` internal self-test simulation rounds, at most `9` LLM requests, and a cumulative `300,000`-token cap. For Codex CLI, standalone `mcp_server.py` enforces identical tool counters and rejects excess calls with `isError: true`.
- **Timeouts, Output Caps, and Lock Budget Decoupling**: Per-task wall time was bounded at `task_timeout = 1800s` during the main campaign (raised to `2400s` by default in the latest codebase revision), with `compiler_timeout = 180s`, `llm_timeout = 420s`, single-turn output caps of `24,576` (Coder/Tester) and `32,768` (Planner/Repair) tokens, and a cumulative fee fuse of `reported_cost_stop_cny = 75`. Recent hardening further decoupled the shared CODESYS mutex wait budget (`compiler_lock_timeout = 960s`, dynamically extended for long-duration timed simu-

lations) from the compiler execution timeout so concurrent workers waiting on the single-instance IDE do not trigger premature lock timeouts.

- **Baseline Context Adaptation (`codex_transport.py`)**: Because Codex CLI injects its own system prompt and tool schemas, applying STAR's tighter input context cap causes immediate prompt truncation; the relay layer relaxes total input context clipping for Codex CLI while preserving identical output token limits, reasoning effort, tool quotas, and stop rules.

3 Benchmark Dataset and Formal Specification Design

3.1 Provenance, Normalization, and Independent Review

The frozen dataset release `normalized-20261114` (SHA-256: `d305f96c2faaa87f60a24be90c00ad0b32153492e741b87547d132c591e0e9ee`) contains **32 control tasks**:

1. **20 Public Tasks (`PUB001`–`PUB020`)**: Ten problems derive from the public course task pool (`PUB001`–`PUB010`) and ten from instructor supplementary tasks (`PUB011`–`PUB020`). Automated alignment (`requirements/source_alignment.json`) against the original 25-page question bank (fingerprint `4ce56562...`) confirms a text sequence similarity of `1.0` and complete interface matching across all 20/20 problems. Ambiguities common in natural-language prompts (concurrent priority order, invalid input sanitization, out-of-bounds clamping, initial output defaults) are clarified additively via `scan_semantics`, `safety_priority`, `assumptions`, and `specification_contract` without modifying the verbatim English statements. All 32 tasks underwent bilingual human verification.
2. **12 Self-Built Tasks (`SELF001`–`SELF012`, AI-Generated + Human Reviewed)**: Marked explicitly with provenance `independent_synthesis`, these tasks were authored through **AI-assisted generation followed by human review**. An AI assistant initially synthesized problem specs, boundary conditions, engineering `rationale`, and 7–12 fixed scan sequences covering multi-actuator interlocks, hysteresis tank control, alternating pump rotation, consecutive-scan qualification, bit-parity checks, and bounded fault retries while avoiding structural duplication of public tasks. Each candidate spec was exercised in Python against an independent reference transition model and then manually audited via a bilingual review view for disambiguation, initialization, priority resolution, recovery edges, and LTL alignment. This review establishes internal benchmark consistency and does not constitute formal IEC 61508 safety certification.

3.2 Industrial Control Semantics and Diversity Metrics

The corpus enforces strict `FUNCTION_BLOCK FB_Task` ownership conventions separating read-only `VAR_INPUT`, `VAR_OUTPUT`, and caller-owned persistent `VAR_IN_OUT` references (e.g., multi-pump banks in `PUB007`, analog channels in `PUB008`, recipe structs in `PUB017`, generic FIFOs in `PUB018`). Timing coverage deliberately combines pure scan-counted discrete state machines across `SELF001`–`SELF012` (immune to host scheduling jitter) with 6 real-timer public problems (`PUB004`, `PUB005`, `PUB006`, `PUB010`, `PUB012`, `PUB013`) totaling 315 non-zero `dt_ms` scan intervals. Table 3 and Figure 1 summarize benchmark breadth across difficulty tiers, control patterns, IEC 61131-3 types, and test-trace scale.

3.3 Scan-Aligned LTL Corpus and Dual-Control SMV Verification

The benchmark incorporates **294 annotated LTL properties** (181 public, 113 self-built) partitioned strictly into two scopes:

- `INITIAL_INSTANCE` **(30 formulas, task `P_init`)**: Constraints checked prior to scan 0 immediately after function-block instantiation (e.g., outputs and retained latches initialized to `FALSE`), preventing erroneous application of temporal operator **G** across pre-execution states.
- `SCAN_SEQUENCE` **(264 formulas)**: Encompasses 50 explicit `safety` invariants, transition dynamics, persistent fault latching (expressed via weak-until expansions such as $\mathbf{G}\, Fault \vee (Fault\ \mathbf{U}\ (\neg Jam \wedge$

Table 3: Diversity profile of the normalized 32-task IEC 61131-3 benchmark (`normalized-20261114`).

Dimension	Measured Coverage	Representative Tasks
Source & Difficulty	32 tasks (20 `public`, 12 `self_built`); Easy 10 (6+4), Medium 14 (8+6), Hard 8 (6+2)	Easy: `PUB001`, `SELF001`; Med: `PUB008`, `SELF007`; Hard: `PUB015`–`020`, `SELF011`–`012`
Control Taxonomy	Step sequencers, multi-mode arbitration, edge bit-counting, debounced/accumulated timers, moving averages, pump alternation, string parsing, struct CRUD, generic FIFOs	`PUB002` (edges), `PUB009` (filter), `PUB013` (HVAC), `PUB017` (recipe), `PUB018` (FIFO), `PUB019` (arm)
IEC 61131-3 Types	21 distinct type forms over 260 declared ports: 205 `BOOL` plus 55 non-Boolean ports across 16 type families (`REAL` 10, `USINT` 9, `WORD`/`TIME`/`ARRAY`/`INT` 4 ea., `DWORD`/`DINT` 3 ea., `LTIME`/`UINT`/`ARRAY[*]`/`LREAL`/`STRUCT`/`VARIANT` 2 ea., `STRING`/`UDINT` 1 ea.)	`PUB008`/`009` (`ARRAY[*]`), `PUB004` (`LTIME`), `PUB014` (`STRING`), `PUB017`/`018` (`VARIANT`)
I/O & Fixed Traces	I/O span 1/1 to 18/6; **276 fixed cases / 2,994 scan steps** (public 177 cases / 2,230 scans; self-built 99 / 764); mean 8.63 cases & 93.56 scans/task	Min I/O: `PUB014` (1/1); Max I/O: `PUB019` (18/6, 17 cases / 306 scans); Longest: `PUB009` (455 scans)

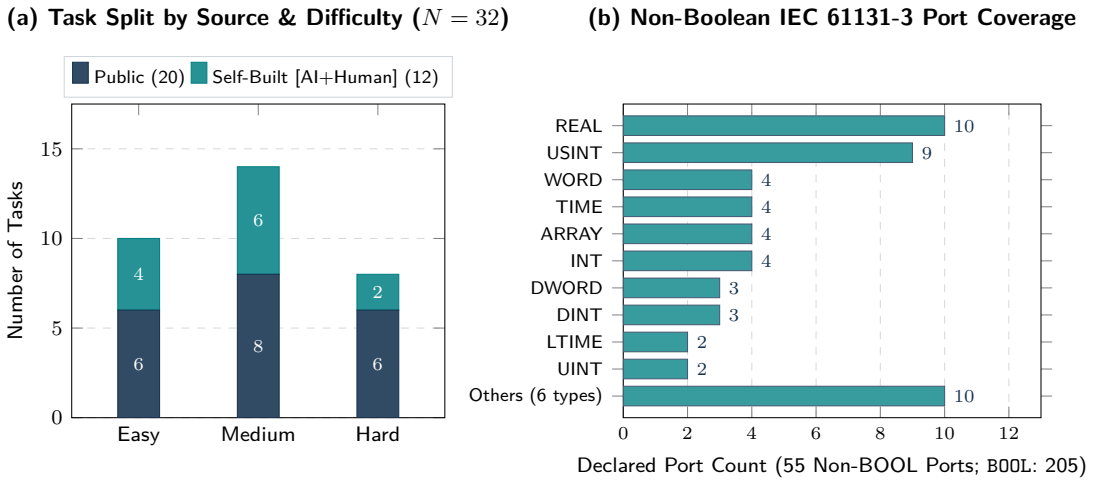


Figure 1: Dataset composition: (a) stratified split across source and difficulty tiers ($N = 32$); (b) coverage of 55 non-Boolean IEC 61131-3 ports (excluding 205 `BOOL` ports) alongside aggregate trace and SMV dual-control counts.

Reset)) in `SELF004` when operator resets may remain idle indefinitely), fairness-guarded conditional liveness, and `CompleteBehavior` specifications.

Whereas Pre-Scan Moore conventions record outputs on the next cycle ($q[k+1]$), real CODESYS cyclic recorders and this benchmark log **End-of-Scan Records**:

$$T[k] = (I[k],\ q[k],\ O[k]), \qquad q[k+1] = \delta(q[k], I[k]), \quad O[k] = \lambda(q[k], I[k]). \tag{1}$$

Because scan k's sampled output $O[k]$ reflects same-scan combinational overrides immediately, safety stop preemption writes cleanly without next-state delay as $\mathbf{G}((StopBtn \vee EStop) \rightarrow \neg Motor)$, while cross-scan retention references explicit pre-state propositions (e.g., `qRun`), keeping discrete runtime traces and SMV models semantically locked.

To verify both specification soundness and fault sensitivity, every task ships an explicit reference SMV model (`models/*_reference.smv`) alongside an adversarial mutant SMV injecting realistic logical or timing bugs (`models/*_mutant.smv`). Tracked under SHA-256 freshness guarantees by `smv_status.py` in `configs/smv_verification_status.json`, **32/32 reference models achieve** `MODEL_PROPERTIES_TRUE` (15 `NuSMV` BDD + 17 `nuXmv` SMT; 285/285 explicit reference properties verified), while **32/32 negative-control mutant models produce** `COUNTEREXAMPLE` (15 BDD + 17 SMT), plus 1 verified `complete` BDD model.

4 The STAR Agent Framework and Supporting Infrastructure

4.1 Overview and Leak-Free Architecture

STAR replaces unstructured free-form agent loops with an artifact-centric multi-stage pipeline operating strictly on a physically scrubbed `agent_view()` input (Figure 2). Every benchmark task strips reference implementations, `tests`, `test_bindings`, `specification_contract`, and `CompleteBehavior` entries prior to exposing prompts to either STAR or Codex CLI, preserving symmetric zero-leakage conditions until offline evaluation in `batch.py`.

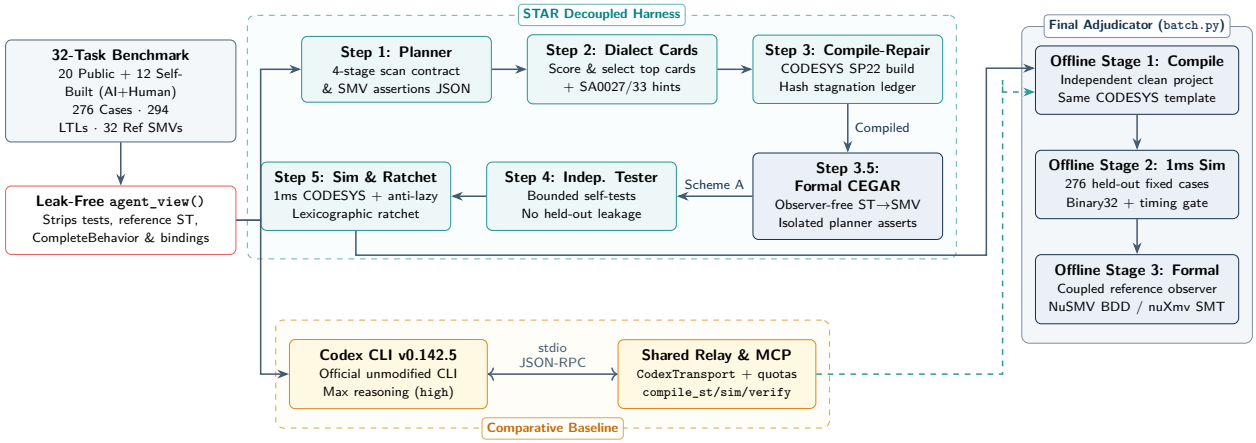


Figure 2: End-to-end architecture comparing the leak-free STAR decoupled pipeline, the symmetric `Codex CLI~v0.142.5 + MCP` baseline, and the three-stage offline adjudicator (`batch.py`).

4.2 Core Design Elements and Engineering Trade-offs

1. **Task Organization (Step 1 Planner)**: Rather than mixing state-machine partitioning with low-level ST syntax in one generation turn, Step 1 emits a structured JSON scan blueprint specifying state enums, timer/edge instances with reset conditions, priority layers, initial values, and a four-phase deterministic scan skeleton: `1) input and edge sampling` → `2) safety preemption`

and state transitions → 3) unconditional top-level TON/R_TRIG execution → 4) post-state output assignment. Step 3 guarantees timers never freeze inside skipped CASE/IF arms, while Steps 2 and 4 prevent intra-scan safety-override conflicts.

2. **Context Management Trade-offs (task_view.py, dialect_cards.py, harness.py)**:
   - *Selective Dialect Cards*: Injecting the complete CODESYS manual wastes context budget and dilutes attention on control logic, whereas omitting dialect nuances invites repeated compiler faults. As a practical trade-off between guidance completeness and prompt focus, STAR curates 8 concise dialect cards (VARIANT, ARRAY[*], LTIME, unconditional TON/R_TRIG, VAR_IN_OUT structs, 0-based s[i] vs. 1-based STRING library indexing, and pre-CASE mode normalization) alongside static-analysis hints, ranking them against interface/text cues and keeping the top-ranked subset (currently top 4).
   - *Diagnostic Compaction & Stagnation Ledger*: compact_diagnostics() extracts localized code snippets and line/column pointers around compiler errors, attaches actionable advice for SA0027 (keyword shadowing) and SA0033 (unused declarations), and pairs with a bounded sliding-window failure log (attempt_ledger) to curb A↔B repair oscillation without unbounded prompt growth.
   - *First-Divergence Desensitized Feedback*: Both simulation and formal loops feed back only the minimal input prefix up to the first failing scan (first_bad_step) or shortest counterexample trace.
3. **Real Compiler Verification & Five-Stage ST->SMV Engine (compiler.py, st_verifier.py)**:
   - *CODESYS Rebuild & 1ms Replay*: Non-ASCII workspace paths are automatically relocated into ASCII staging directories, builds verify app.generate_code() static checks, nonces, and template SHA-256 digests, and simulation tests execute isolated FB_Task test programs via binary32 byte-exact float matching to avoid JSON rounding drift.
   - *Observer-Free In-Loop Verification (Scheme A) & Expression Lowering*: Inside Step 3.5 (and baseline MCP verify_st), st_verifier.py parses ST ASTs into pre-init ($t = -1$) and single-scan SSA transition models ($t \geq 0$) with qualified enum resolution and built-in symbolic semantics for 9 standard FBs (TON, TOF, TP, R_TRIG, F_TRIG, RS, SR, CTU, CTD). Built-in _SMVExprParser and _translate_st_ltl_to_smv automatically normalize mixed Boolean/relational operators, unfold chained comparisons (e.g., -4 <= x <= 4), translate typed base literals (DWORD#0, WORD#16#...), and reject malformed nested temporal expressions prior to invoking BDD/SMT solvers. Crucially, internal generation runs Scheme A without linking the held-out reference observer (include_expected_observer=False), while keeping Planner-generated formal_assertions isolated so malformed planner assertions automatically fall back to benchmark LTLs rather than halting verification. Only frozen final candidates enter Stage 3 coupled reference-observer equivalence checks. Unsupported constructs (e.g., VARIANT, __VARINFO) cleanly report UNSUPPORTED_ST alongside PASS, COUNTEREXAMPLE, INCONCLUSIVE, TIMEOUT, and TOOL_ERROR.
4. **Repair Loop Engineering Trade-offs & Termination**: When Step 4 constructs self-tests without oracle ground truth, forbidding all test pruning risks misleading repairs from faulty self-tests, while unrestricted pruning risks trivializing tests. STAR enforces a structural constraint—lazy_rejection_blocked—disallowing case dismissal unless the candidate code hash substantively changes. Candidate acceptance follows a lexicographic 4-tuple (compiled, formal_passed_props, sim_passed_cases, -len(rejected_case_ids)): if a candidate edit regresses any higher-priority criterion, the ratchet rolls back to the best verified version. The trade-off is favoring reliable local convergence under tight repair budgets over multi-step disruptive refactoring. Finally, infrastructure faults (TOOL_ERROR, TIMEOUT, UNVERIFIED) remain cleanly separated from logical outcomes.

4.3 Specification Auditing Support and Zero-Dependency WebUI Visualization

Beyond core synthesis, engineering effort invested substantially in making formal specifications auditable, regressions reproducible offline, and agent traces visually inspectable:

4.3.1 Multi-Layer Specification Auditing and Offline Regression

1. `ScanContract` **Dual-Mode Offline Regression**: Combines transition-model multi-step contract validation and differential random testing so the core timing and semantic suite runs fast on headless Linux nodes without Windows/CODESYS dependencies.
2. **Three-Variant SMV Freshness Tracking (`smv_status.py`)**: Automatically discovers `reference`, `complete`, and `mutant` SMV files per task, infers `bdd` vs. `smt` solver requirements, probes `NuSMV`/`nuXmv` binary availability, and hashes model/config state into `configs/smv_verification_status.json`, flagging edited artifacts automatically as `STALE` (or `CHECKER_MISSING`/`UNVERIFIED`) so outdated proofs are never reused.
3. **Six-Category LTL Symbol Taxonomy (`symbol_glossary.py`)**: Parses every identifier appearing inside LTL formulas and contracts and assigns it into six semantic classes—`input` ($I[k]$), `output` ($O[k]$), `persistent_inout` (caller-owned `VAR_IN_OUT`), `obs_history` (prior-scan output $O[k-1]$), `ghost_state` (cross-scan reference state, edge memory $I[k-1]$, timer internals), and `alias` (combinational/temporal/contract macro aliases)—paired with bilingual definitions to eliminate observer-variable ambiguity during audits.

4.3.2 Full-Stack Bilingual Industrial WebUI (`webui.py` & `run_trace.py`)

Implemented strictly over Python standard-library HTTP services with zero third-party frontend/backend dependencies, the WebUI unifies live backend status diagnostics and four specialized workspaces:

1. **Dataset & Specs (`tab-explorer`)**: Supports source/difficulty filtering across all 32 tasks, side-by-side bilingual spec inspection, nested-bracket LTL highlighting with click-to-inspect six-class definitions (`spec-popover` powered by `symbol_glossary.py`), reference ST/`.smv` inspection, and live in-browser `NuSMV`/`nuXmv` re-verification via `/api/check-reference-model`.
2. **Agent Workflow (`tab-trace` + `run_trace.py`)**: Safely resolves workspace artifacts inside `experiments/` and `results/raw/` to reconstruct complete execution timelines—displaying Planner contract JSON, Dialect card selections, per-turn `version_*` syntax diffs, compact diagnostics, and CEGAR/simulation repair iterations.
3. **Sandbox & Waveform Lab (`tab-sandbox`)**: Provides an interactive ST editor with live demo chips connected to `/api/evaluate`, `/api/verify-candidate`, and `/api/export-verify`, plus `_build_step_waveforms()` to plot digital square waves and analog step trajectories while highlighting the exact *first-divergence scan* where candidate behavior deviates from expected traces.
4. **Benchmark Results (`tab-benchmark`)**: Aggregates multi-condition experiment summaries via `/api/experiments` across overall, public/self-built, and difficulty splits (corresponding directly to the aggregate metrics in Section 5.2).

5 Empirical Validation, Benchmark Results, and Analysis

This section reports baseline infrastructure and reference-model preflight validation (Section 5.1), complete quantitative benchmark results across the 2×2 crossed 128-run matrix (**E1**–**E4**, 32 tasks each; Section 5.2), cross-condition diagnostic case studies (Section 5.3), and methodological threats to validity (Section 5.4).

5.1 Infrastructure and Reference-Model Preflight

Prior to multi-condition LLM benchmarking, every static validator, toolchain adapter, and reference oracle underwent isolated verification (Table 4). Because `tests/test_webui.py` evolves alongside active UI refinements, core regression checks cover the 27 non-WebUI Python test modules (`223/223` passing assertions on current `HEAD`).

Table 4: Pre-campaign infrastructure, compiler, static-analysis, and reference-model validation summary.

Verification Check	Command / Evidence	Observed Outcome
Dataset Integrity	`plc-agent validate-dataset`	32 tasks (20 public + 12 AI-generated/human-reviewed), 276 cases, 2,994 scans, 294 LTLs; `errors=[]`; 1.0 text alignment on 20/20 public tasks.
Core Unit Test Suite	`unittest` (27 non-WebUI modules)	**223/223 passed** covering view isolation, relay/budget quotas, decoupled lock timeouts, `ST->SMV` lowering, and MCP tooling.
CODESYS Compile & SA Probe	`compiler_preflight.json`, `static_analysis_preflight.json`	Verified `PASS (20.3s)` → fault injection `C0032/C0046` (`COMPILE_FAIL`) → recovery `PASS (24.7s)`, plus `SA0033` gating.
Reference & Mutant SMVs	`configs/smv_verification_status.json`	**32/32** reference SMVs `MODEL_PROPERTIES_TRUE` (15 `NuSMV` BDD + 17 `nuXmv` SMT; **285/285** properties) and **32/32** mutants `COUNTEREXAMPLE`.
Reference `ST->SMV`	`st_formal_verification.json`	**14/14 `PASS`** on supported fragment (all 12 `SELF` + `PUB006/007`); 2 tasks with `VARIANT` classified `UNSUPPORTED_ST`.
`ScanContract` Regression	`scan_contract_acceptance.json`	44 contract unit checks + 15 full-task transition verifications passed; **6/6** injected mutant traces detected as `VIOLATED`.

5.2 Full Four-Condition Matrix Results: E1–E4 Across 128 Runs

Drawing on the 14 November 2026 unified snapshot (`results/summary/fullflow_matrix_snapshot_20261114.json`), resumed `Codex CLI` records (`results/raw/codex_matrix_resume_20261114/`), and recovered **E3** workspaces (`results/raw/six_task_fullflow_20261114/`), all four conditions (**E1**–**E4**) have completed end-to-end synthesis, CODESYS V3.5 SP22 compilation, real `1ms` simulation replay, and Stage 3 coupled `ST->SMV` adjudication across all 32 tasks (128/128 runs) under maximum reasoning effort (`reasoning.effort = "high"`). Table 5 strictly separates *evaluated* denominators from *planned* denominators (276 fixed cases across 32 tasks) and retains pre-`cef1388` `TOOL_ERROR` and `TIMEOUT` events unaltered.

5.2.1 Visualizing Empirical Trade-Offs and Key Observations

Figures 3–5 dissect the empirical profile across all four experimental conditions (**E1**–**E4**, 128 runs total) by aggregate split metrics, difficulty scaling, Stage 3 formal outcomes, and generation efficiency.

Four consistent empirical findings emerge from Table 5 and Figures 3–5:

Table 5: Complete four-condition empirical benchmark summary across 128 runs (32 tasks per condition, as of 14 November 2026).

Evaluation Metric	E1: Codex + DeepSeek	E2: Codex + Terra	E3: STAR + DeepSeek	E4: STAR + Terra
Completed Tasks	**32/32** (uniform audit)	**32/32** (uniform audit)	**32/32** (uniform audit)	**32/32** (uniform audit)
First-Pass Compile	23/32 (71.9%); +5 repaired (18 repairs, 50 calls)	26/32 (81.2%); +3 repaired (4 repairs, 36 calls)	**27/32 (84.4%)**; +5 repaired (5 repairs, 40 calls)	**27/32 (84.4%)**; +1 repaired (3 repairs, 39 calls)
Final Compilation (All / SELF / PUB)	**28/32 (87.5%)** (S 11/12, P 17/20; 4 SA0027)	**29/32 (90.6%)** (S 12/12, P 17/20; 1 FAIL, 2 TE)	**32/32 (100.0%)** (S 12/12, P 20/20)	**28/32 (87.5%)** (S 12/12, P 16/20; 1 FAIL, 3 TE)
Fixed Cases (Eval. Denom.)	**196/208 (94.2%)** (S 92/92, P 104/116)	**220/222 (99.1%)** (S 99/99, P 121/123)	**226/238 (95.0%)** (S 99/99, P 127/139)	**217/221 (98.2%)** (S 99/99, P 118/122)
Fixed Cases (Planned 276)	**196/276 (71.0%)**	**220/276 (79.7%)**	**226/276 (81.9%)**	**217/276 (78.6%)**
Task Functional Pass (All / Eval.)	**23/32 (71.9%)**; 23/27 (85.2%) (S 11/12, P 12/20)	**27/32 (84.4%)**; **27/29 (93.1%)** (S 12/12, P 15/20)	**27/32 (84.4%)**; 27/31 (87.1%) (S 12/12, P 15/20)	**25/32 (78.1%)**; 25/28 (89.3%) (S 12/12, P 13/20)
Tier Task Pass (E10/M14/H8)	7/10 (70.0%) · 11/14 (78.6%) · 5/8 (62.5%)	10/10 (100.0%) · 13/14 (92.9%) · 4/8 (50.0%)	9/10 (90.0%) · 13/14 (92.9%) · 5/8 (62.5%)	8/10 (80.0%) · 13/14 (92.9%) · 4/8 (50.0%)
Stage 3 Formal (ST->SMV)[a]	PASS 16 (S11+P5); CE 3; TO 1; UNSUP 8; NR 4	PASS **18** (S12+P6); CE 1; TO 1; UNSUP 9; NR 3	PASS 17 (S12+P5); CE 3; UNSUP 12; TO/NR 0	PASS **18** (S12+P6); CE 1; TO 1; UNSUP 8; NR 4
Adjudicator Verdicts (batch.py)	PARTIAL 20, FAIL 10, TO 1, TE 1	PARTIAL 26, FAIL 3, TO 1, TE 2	PARTIAL 26, FAIL 5, TE 1	PARTIAL 24, FAIL 4, TO 1, TE 3
Gen. Calls & In-Loop Tools	184 reqs (50 compile / 9 sim / 37 verify)	93 reqs[b] (36 compile / 1 sim / 1 verify)	104 reqs (40 compile / 34 sim / 32 verify)	99 reqs (39 compile / 32 sim / 28 verify)
Tokens (32/32 Tasks)	4,742,114 in (85.6% cached) / 515,786 out (331,389 reas.)	1,260,812 in (58.9% cached) / **72,823 out** (20,979 reas.)	432,953 in (17.3% cached) / 778,596 out (651,225 reas.)	**365,578 in** (11.2% cached) / 215,832 out (125,818 reas.)
API Cost & Gen. Latency[b]	**¥2.15**; mean 143.5 s (med. 107.0 s)	**¥5.21**; mean **91.0 s** (med. 66.1 s)	**¥2.74**; mean 218.2 s (med. 171.5 s)	**¥7.38**; mean 129.2 s (med. 98.1 s)

[a] Abbreviations: S/P = SELF/PUB; CE = COUNTEREXAMPLE; TO = TIMEOUT; TE = TOOL_ERROR; UNSUP = UNSUPPORTED_ST; NR = NOT_RUN. Timed test cases exceeding practical wall-clock bounds or zero-tolerance host timing limits, plus uncompiled or harness-incompatible tasks, are excluded from evaluated-denominator percentages rather than marked as passes.
[b] Generation telemetry covers all 32 tasks per group (**E3** includes the 6 recovered tasks under `results/raw/six_task_fullflow_20261114/`, totaling 104 requests and ¥2.74). Provider cost is absent on 1 of 184 requests in **E1**, and token/cost usage fields are absent on 10 of 93 requests in **E2**.

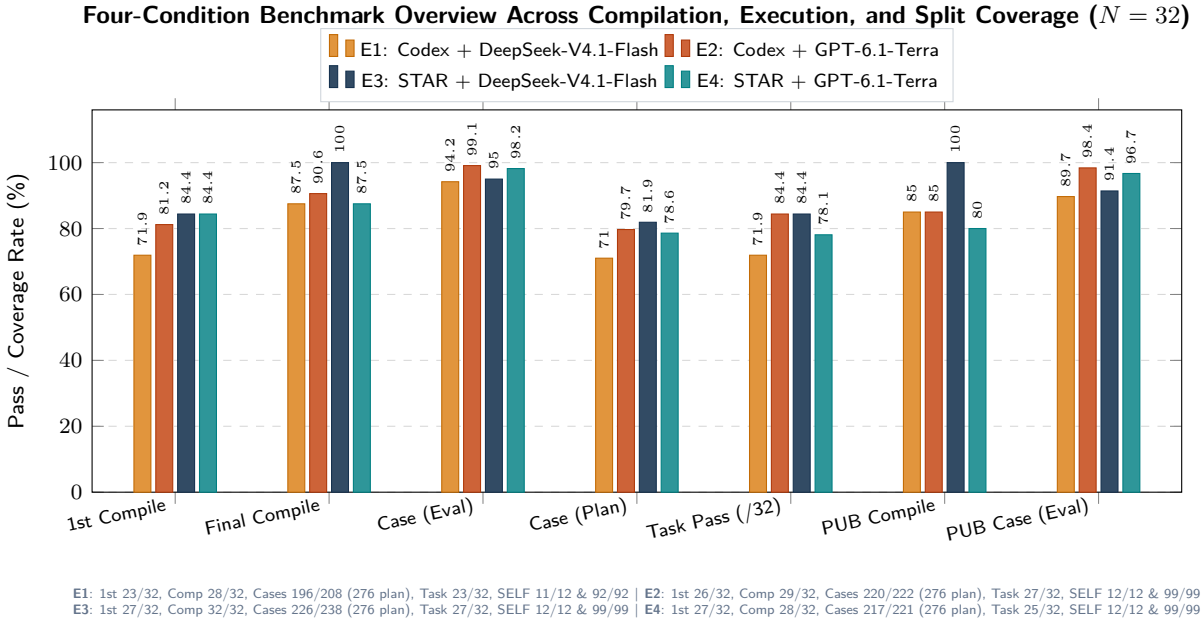


Figure 3: Cross-condition comparison across **E1** (Codex + DeepSeek), **E2** (Codex + Terra), **E3** (STAR + DeepSeek), and **E4** (STAR + Terra) covering first-pass and final compilation, evaluated vs. planned fixed cases, task-level pass rate, and `public` ($n = 20$) vs. `self_built` ($n = 12$) splits.

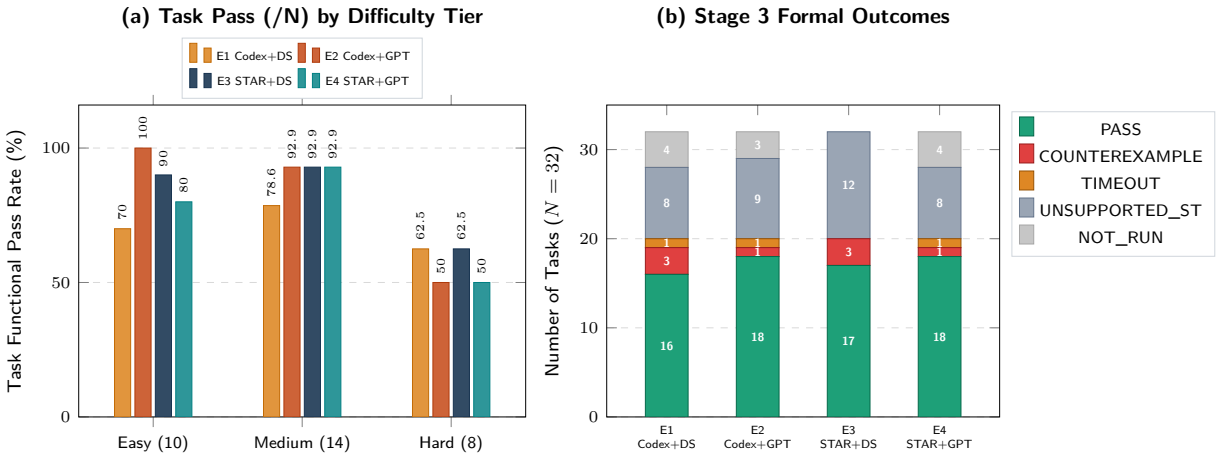


Figure 4: Stratified performance and formal diagnosis across **E1**–**E4**: (a) task-level functional pass rate across Easy ($n = 10$), Medium ($n = 14$), and Hard ($n = 8$) tiers; (b) Stage 3 formal `ST->SMV` outcome counts ($N = 32$ tasks per condition).

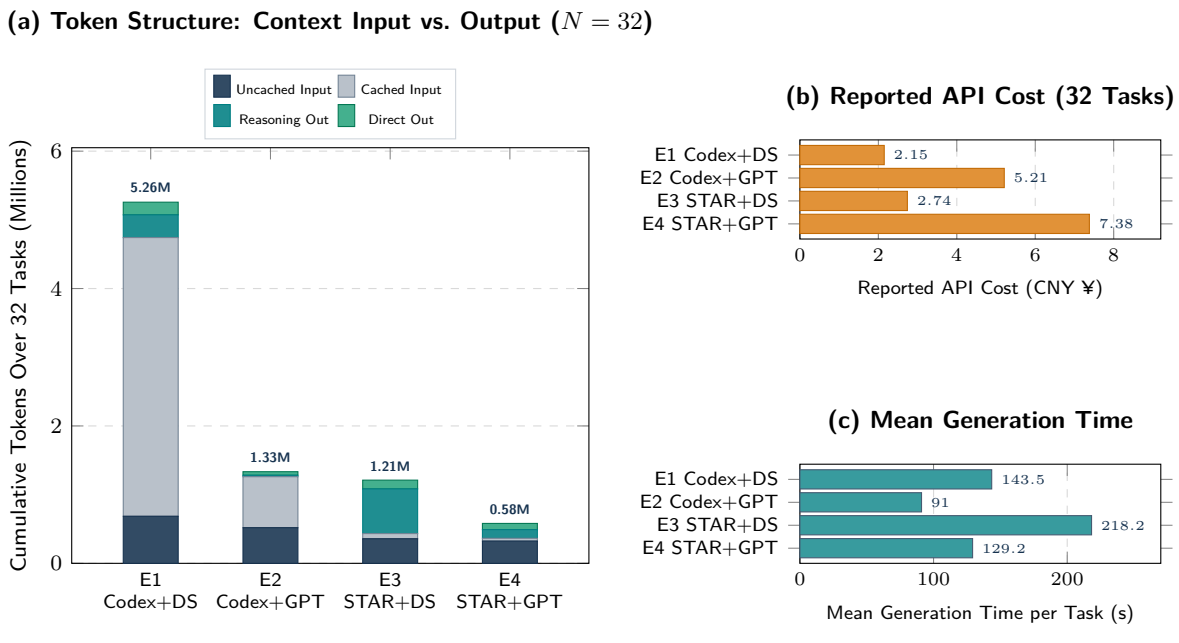


Figure 5: Generation resource efficiency across **E1**–**E4** ($N = 32$ tasks per condition): (a) cumulative 32-task input tokens (cached vs. uncached) and output tokens (reasoning vs. direct emission); (b) total reported provider API cost (RMB); (c) mean generation elapsed time per task (s).

1. **Framework Benefit Depends Sharply on Foundation Model Capability (E3 vs. E1)**: On `deepseek-v4.1-flash-1017`, unguided `Codex CLI` (**E1**) issues 184 LLM requests and 50 compiles, accumulating 4,742,114 input tokens (4,057,344 cached, 684,770 uncached) across multi-turn agent transcripts, yet stalls at **28/32 (87.5%)** compilation, 196/208 (94.2% evaluated; 71.0% planned) fixed cases, and 23/32 (71.9%) task passes with 10 functional failures. Under **STAR** (**E3**), view isolation, targeted dialect cards, and compact diagnostics raise compilation to **32/32 (100.0%)**, lift evaluated cases to 226/238 (and peak planned coverage at **81.9%**), boost task passes to 27/32 (halving `FAIL` outcomes to 5), and compress total input volume roughly **11**× to 432,953 tokens (104 requests).
2. **Structured Multi-Role Pipeline vs. Unconstrained Tool-Use on Stronger Models (E2 vs. E4)**: On `gpt-6.1-terra`, unconstrained `Codex CLI` (**E2**) rarely requires exploratory recovery (triggering only 1 in-loop simulation and 1 formal call across all 32 tasks), achieving **29/32 (90.6%)** compilation, the matrix-high **99.1% (220/222)** evaluated case pass rate, and **27/32** task passes (27/29 = 93.1% evaluated) with minimal 72,823 total output tokens, ¥5.21 cost, and 91.0 s mean generation latency. Under **STAR** (**E4**), the model achieves the same **18** Stage 3 formal `PASS` verdicts and lowest input footprint (365,578 tokens), but mandatory multi-stage JSON Planning, Coding, and Testing inflate total output tokens nearly 3× (215,832), raising cost to ¥7.38 and mean latency to 129.2 s while passing two fewer tasks overall (one extra `FAIL` on `PUB004` and one extra lock timeout on `PUB006`). Fixed heavy orchestration therefore adds output overhead on models already strong enough to compile cleanly, motivating adaptive difficulty-gated routing.
3. **High Reliability on Self-Built Control Suite (`SELF001`–`SELF012`)**: Across **E2**, **E3**, and **E4**, all three conditions achieve **12/12 compilation, 99/99 fixed-case pass, and 12/12 Stage 3 formal verification pass**. Across all 48 self-built runs in the 2 × 2 matrix, only one run failed—**E1**/`SELF009`—solely due to omitting CODESYS static-analysis dialect guardrails (`SA0027`).
4. **Lock Contention Dominates GPT Compilation Gaps and `PARTIAL` Bounds Reflect Execution Scope**: Beyond `PUB018` (`VARIANT` reflection), all uncompiled tasks in **E2** (`PUB014`, `PUB017`) and **E4** (`PUB006`, `PUB014`, `PUB017`) terminated as infrastructure `TOOL_ERROR`s from compiler mutex contention (`Compiler lock wait deadline exceeded`), directly prompting commit `cef1388` to decouple lock wait deadlines from compile timeouts. Separately, `PARTIAL` verdicts in `batch.py` stem from physical hour-scale timer exclusions and explicit `UNSUPPORTED_ST` lowering exits rather than silent falsification.

5.3 Deep-Dive Case Studies

5.3.1 Case Study 1: Hidden Single-Scan Delay Masked by 100% Simulation Pass (`PUB011`)

`PUB011` specifies a three-station transfer trolley toggling between manual and automatic operation. Cross-condition inspection contrasts both runtime-vs.-formal coverage and per-run synthesis variance:

- **E3 Passes 8/8 Fixed Tests Yet Fails 8/14 NuSMV Properties**: DeepSeek under STAR (`results/raw/four_group_20261114/star_deepseek_v3/PUB011/generation/compile_00/FB_Task.st`) achieves **8/8 (100%)** pass on real CODESYS 1 ms simulation tests. During Stage 3 verification, however, `NuSMV` proves only 6/14 LTLs and builds deterministic counterexamples against 8 transition obligations (incl. `SMV_4`, `SMV_8`, `SMV_9`) due to an intermediate `E_Phase.MANUAL` state inserted before the dispatcher: On the exact rising-edge scan transitioning from `Mode = FALSE` to `TRUE` with `SensorA` and `ButtonA` asserted, one scan is spent resetting `phase` from `MANUAL` to `WAIT_A`, delaying `MotorForward` by one scan cycle—a corner case never triggered by the 8 finite test sequences.
- **E1, E2, and E4 Preserve Same-Scan Entry Semantics**: By contrast, **E4** (`results/raw/four_group_20261114/star_terra/PUB011/generation/compile_00/FB_Task.st`, maintaining `E_TaskPhase.WAIT_A` during `NOT Mode`) as well as both `Codex CLI` baselines (**E1** and **E2**) stay directly in their `WAIT_A` phase while in manual mode, allowing immediate same-scan transition and

Listing 1: Relevant E3 (PUB011) mode-switch logic introducing a one-scan startup delay.

```
IF NOT Mode THEN
    phase := E_Phase.MANUAL;
ELSIF phase = E_Phase.MANUAL THEN
    phase := E_Phase.WAIT_A;
ELSE
    CASE phase OF ...
```

simultaneously passing **8/8 simulation cases and 14/14 formal LTLs**.

5.3.2 Case Study 2: Universal Transient Latching and Priority Arbitrations (PUB019)

On the 18-input, 6-output multi-mode robotic arm controller PUB019 (Hard), **all four conditions (E1–E4) converge on the exact same residual flaw**: every group scores **16/17** on fixed cases, failing exclusively at scan 1 of PUB019.T6 (MoveUp expected TRUE, measured FALSE), and every group terminates with Stage 3 COUNTEREXAMPLE (**E1**: 4/8, **E2**: 3/8, **E3**: 5/10, **E4**: 6/10 properties satisfied). Two common failure mechanisms explain this convergence:

1. **Over-Suppression on Entry From Idle**: At scan 0 of PUB019.T6, no mode is active; scan 1 raises ManualMode alongside UpButton. Across all four conditions, models detect modeChanged against quiescent idle state and blank actuator outputs for one scan, rather than gating only active-to-active mode switches.
2. **Conservative Mutual Lockout vs. Manual Preemption**: When ManualMode and an auto-mode selector activate concurrently, all four candidates apply modeCount > 1 to freeze all axes safely, whereas the specification's observer defines explicit manual override precedence, triggering immediate scan-0 formal counterexamples.

5.3.3 Case Study 3: Formal Catch on Bare Codex (PUB020), Model Divergence (PUB016), and Indexing Nuance (PUB014)

- PUB020 **(Multi-Station Production Line)**: Both gpt-6.1-terra conditions (**E2** and **E4**) pass **11/11 fixed cases and Stage 3** PASS. Conversely, both DeepSeek runs draw Stage 3 counterexamples: critically, **E1** passes **all 11/11 finite simulation cases** yet satisfies only 3/11 NuSMV properties (exposing mutual-exclusion violations in REJECT reachable states, complementing **E3**/PUB011), while **E3** drops late progression in PUB020.T9 (10/11 cases, 3/10 formal).
- PUB016 **(Analog Ramp/Step Control)**: Inverting overall model rank order, both DeepSeek groups (**E1** and **E3**) pass **8/8** tests, whereas both GPT groups (**E2** and **E4**) fail identically on PUB016.T4 (**7/8**).
- PUB014 **(Dotted-Decimal IPv4 Parser)**: Both DeepSeek runs (**E1** and **E3**) compile on the first attempt yet score **0/8** at runtime because indexing ipText[i] defaults to 1-based 1..LEN(ipText) instead of CODESYS's **0-based** STRING byte offset s[i], reading past the terminator and clearing output. (Both GPT conditions hit compiler-lock timeouts before simulation.) Targeted updates to dialect_cards.py subsequently codify this 0-based vs. 1-based distinction explicitly.

5.3.4 Case Study 4: Vendor Static-Analysis Guardrails (SA0027/SA0033) and Generic Reflection (PUB018)

- **Same-Root** SA0027 **Elimination Across E1 vs. E3**: All four compilation failures in **E1** (Codex + DeepSeek: PUB003, PUB009, PUB017, SELF009) failed on the exact same CODESYS static-analysis collision—SA0027 (Variable name 'error' in 'FB_Task' already used for an object in Library 'caa device diagnosis, 3.5.22.0'), compounded by SA0033 on PUB017—despite multiple multi-turn repair iterations. By contrast, **E3** (**STAR** + DeepSeek) compiled **4/4 with**

zero SA0027 failures (three on first attempt, PUB017 after one repair) via dialect cards and compact_diagnostics(), while **E2** (Codex + Terra) resolved the first three after 1–2 iterative fixes.

- PUB018 **(Generic Circular Byte Queue)**: Binding caller-owned DINT variables and fixed arrays via VAR_IN_OUT VARIANT / VAR_INFO marks an infrastructure and abstraction frontier: GPT conditions (**E2**, **E4**) hit COMPILE_FAIL (Unknown type: 'VARIANT'), whereas DeepSeek conditions (**E1**, **E3**) compile cleanly yet incur external harness TOOL_ERROR during simulation type binding.

5.4 Threats to Validity

Three methodological boundaries frame interpretation of the empirical findings:

1. **Physical Timer Scale and Host Scheduling Jitter**: Several public timer benchmarks specify cumulative holds spanning minutes or hundreds of hours (e.g., PUB004.T3/T5), which cannot run in real wall time within fixed campaign budgets. Conversely, sub-second step intervals executed under Windows host 1 ms task scheduling exhibit microsecond-to-millisecond jitter that can trigger strict tolerance exclusions (TIMING_NOT_REPRODUCED). Reporting separates these excluded timed cases transparently alongside planned denominators rather than introducing unverified synthetic time-scaling factors.
2. **Numeric Semantics Between Real Hardware Types and Exact SMT**: For floating-point tasks such as PUB003, nuXmv/MathSAT models REAL values over exact rationals rather than IEEE-754 single-precision rounding and overflow. In practice, real CODESYS simulation validates byte-exact binary32 outputs, turning the two stages into complementary runtime and symbolic checks.
3. **Symbolic Coverage Frontier and Version Cutoff**: Dynamic VARIANT bindings, computed string byte slices, and unmodeled arithmetic idioms remain outside the current ST->SMV lowering fragment and map cleanly to UNSUPPORTED_ST (8–12 tasks across **E1**–**E4**). Additionally, commit cef1388 (decoupled compiler_lock_timeout=960s and expanded SMV expression lowering) landed after the 14 November 2026 campaign cutoff; this report preserves all original TOOL_ERROR and TIMEOUT classifications verbatim without post-hoc relabeling.

6 Conclusion and Future Work

6.1 Summary of Contributions

Targeting automated IEC 61131-3 ST synthesis on CODESYS V3.5 SP22 Patch 4, this project delivers an integrated engineering suite combining: (1) a normalized 32-task, 276-case, 294-LTL end-of-scan benchmark preserving 20 verbatim public prompts alongside 12 AI-generated, human-reviewed self-built controllers, backed by positive/negative dual-control verification (32/32 reference TRUE; 32/32 mutant COUNTEREXAMPLE); (2) the leak-free **STAR** multi-stage agent framework uniting structured scan planning, selective dialect cards, hash-guarded compiler repair, assertion-isolated in-loop ST->SMV CEGAR verification, and anti-lazy lexicographic ratchet selection; and (3) a symmetric Codex CLI v0.142.5 + MCP baseline harness alongside offline ScanContract verification, six-category symbol_glossary.py auditing, and a zero-dependency interactive WebUI.

Completing the full 2×2 crossed 128-run benchmark (**E1**–**E4**) under maximum reasoning effort reveals crisp architecture–model interactions. For deepseek-v4.1-flash-1017, **STAR** raises compilation from **87.5% to 100.0%**, expands evaluated fixed-case coverage from **196 to 226**, eliminates all SA0027 collisions, and cuts input context **11**×. For gpt-6.1-terra, unconstrained Codex CLI reaches **99.1% (220/222)** evaluated case accuracy at lower reported cost (¥5.21 vs. ¥7.38), whereas **STAR** adds output-token overhead without functional gain once compilation succeeds cleanly. Meanwhile, concrete counterexamples hidden behind 100% passing simulation suites on both **E3**/PUB011 and **E1**/PUB020 confirm the indispensable value of independent symbolic model checking.

6.2 Immediate Next Steps

1. **Post-`cef1388` Validation Rerun, Paired Statistics, and Module Ablations**: Re-evaluate lock-contended tasks under the decoupled 960 s mutex budget and enhanced expression lowering (reported in a separate post-cutoff table), apply exact McNemar paired tests and 95% bootstrap CIs across all 128 runs, run `no_planner`, `no_dialect`, and `no_selftest` ablations, and prototype complexity-gated routing to avoid multi-role output tax on straightforward tasks.
2. **Expand Symbolic Coverage and Multi-Vendor PLC Scope**: Extend `ST->SMV` support toward integer modulo and bounded array/string slicing idioms, and generalize the decoupled compiler and simulation backend contract toward additional multi-vendor PLC toolchains such as Siemens TIA Portal.

References

[1] CODESYS Group, *Starting Scripts from the Command Line — CODESYS Scripting Documentation*. `https://content.helpme-codesys.com/en/CODESYS%20Scripting/_cds_starting_script_via_command_line.html`.

[2] CODESYS Group, *ScriptApplication and ScriptSystem Automation Interfaces*. `https://content.helpme-codesys.com/en/ScriptingEngine/ScriptApplication.html`.

[3] A. Cimatti, E. Clarke, E. Giunchiglia, F. Giunchiglia, M. Pistore, M. Roveri, R. Sebastiani, and A. Tacchella, "NuSMV 2: An OpenSource Tool for Symbolic Model Checking," in *CAV*, 2002.

[4] R. Cavada et al., "The nuXmv Symbolic Model Checker," in *CAV*, 2014.

[5] OpenAI, *Codex CLI and Model Context Protocol (MCP) Reference*, v0.142.5, 2026.